

PROGRAMMING LANGUAGES

GRAMMARS IN ANTLR

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.Y. 2025/2026



© Michele Pasqua, 2026

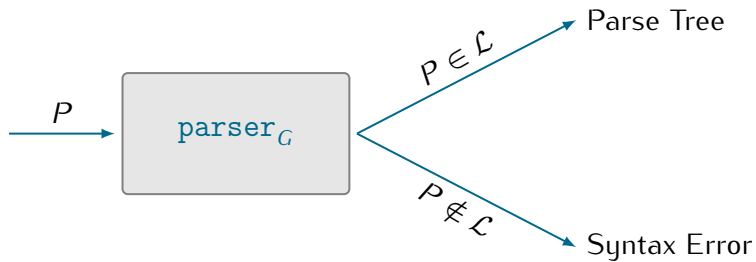
LANGUAGE GRAMMARS



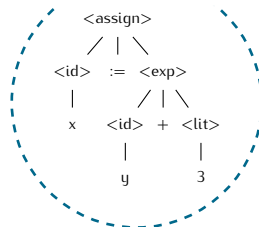
CONTEXT-FREE GRAMMAR

We start from a CF grammar G , generating a language $\mathcal{L}$

- The parser is automatically derived by ANTLR



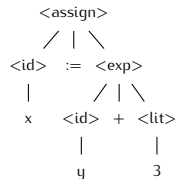
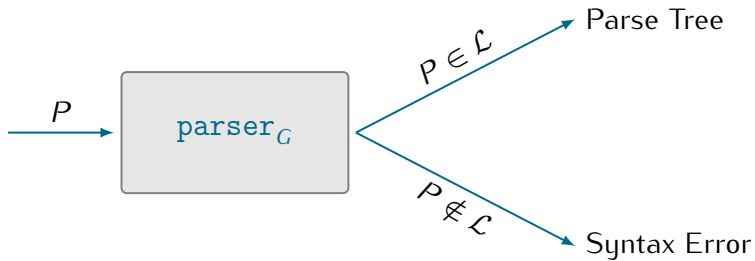
Define semantics



SYNTACTIC ANALYSIS

A parser performs a **syntactic analysis** on the input string

- Returning a syntax error or a parse tree



SYNTACTIC ANALYSIS (CONT'D)

A parser performs a **syntactic analysis** on the input string

- ▶ Returning a syntax error or a parse tree

That consists of two main steps

1. **Tokenization** (Lexical Analysis), grouping characters into words (**tokens**)
2. **Parsing**, recognizing the program structure and producing the parsing tree

TOKENIZATION

```
public int factorial(int n) {  
    int r = 1;  
    for (int i = n; i > 1; --i)  
        r *= i;  
    return r;  
}
```

1
2
3
4
5
6

public

PUBLIC

...

, ,

WS

int

INT

, ,

WS

factorial

ID

(

LPAR

AMBIGUITIES IN LEXICAL ANALYSIS

Multiple matching tokens

- ▶ Come first, match first

FOR : 'for' ;

ID : [a-z]+ ;



Matching sub-tokens

- ▶ Longest-match rule

A : 'a' ;

B : 'b' ;

AB : 'ab' ;



PARSING AMBIGUITY

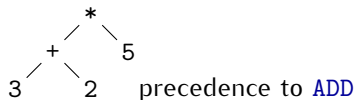
A grammar is **ambiguous** if a string can have more than one parse tree

```
exp : NAT | exp ADD exp | exp MUL exp ;
```

```
ADD : '+' ;
```

```
MUL : '*' ;
```

```
NAT : '0' | [1-9][0-9]* ;
```



Which is the parse tree for 3+2*5 ?

// *wrong output: 25*

► First-matching rule, from left to right **ANTLR**

GRAMMAR GOLD STANDARD

No indirect left recursion

$S \rightarrow aS$



$S \rightarrow Sa \mid b$



$S \rightarrow S$



$S \rightarrow T$

$T \rightarrow S$



$S \rightarrow T$

$T \rightarrow S \mid aS$



GRAMMAR GOLD STANDARD (CONT'D)

It may be a problem for top-down parsers

- ▶ They may end up in infinite recursive cycles

Indirect left recursion can be algorithmically removed

- ▶ ANTLR assumes no left-recursion

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

Therapist: Infinite recursion meme can't hurt you

Infinite recursion meme:

GRAMMARS IN ANTLR



LISTS OF DIGITS

Given the alphabet $\text{Dig} = \{0, 1, \dots, 9\}$

Define a context-free grammar to generate the language

$$\mathcal{L}_{\text{Dig}} = \{(d_1, d_2, \dots, d_n) \mid n \in \mathbb{N} \wedge \bigwedge_{0 < i \leq n} d_i \in \text{Dig}\}$$

of all lists of decimal digits

! Note that, $() \in \mathcal{L}_{\text{Dig}}$

// case $n = 0$

RECURSIVE LISTS OF DIGITS

Now consider, for all $k \in \mathbb{N}$, the following recursive definition

$$\begin{cases} \mathcal{L}_{\text{Dig}}^k = \text{Dig} & \text{if } k = 0 \\ \mathcal{L}_{\text{Dig}}^k = \left\{ (l_1, l_2, \dots, l_n) \mid n \in \mathbb{N} \wedge \bigwedge_{0 < i \leq n} l_i \in \bigcup_{0 \leq j \leq k} \mathcal{L}_{\text{Dig}}^{j-1} \right\} & \text{if } k > 0 \end{cases}$$

Define a context-free grammar to generate the language

$$\mathcal{L}_{\text{Dig}}^* = \bigcup_{i \geq 0} \mathcal{L}_{\text{Dig}}^i$$

of all recursive lists of decimal digits

// e.g., $(2, ((8), 3), ()) \in \mathcal{L}_{\text{Dig}}^*$

ANTLR IMPLEMENTATION

- ✘ Implement the grammar for $\mathcal{L}_{\text{Dig}}$ and $\mathcal{L}_{\text{Dig}}^*$ in ANTLR
 - ▶ Do not use indirect left recursion
- ✘ Implement a simple parser for the two grammars
 - ▶ Write test cases for recognizable and not recognizable strings

MORE GRAMMARS

✘ Implement a CF grammar for $// 011011, 11111 \in \mathcal{L}_1 \text{ but } 0101, 111 \notin \mathcal{L}_1$

$$\mathcal{L}_1 = \{w \in \{0, 1\}^* \mid w \text{ contains at least four } 1\}$$

✘ Implement a CF grammar for $// 01b, 0_1_01b \in \mathcal{L}_2 \text{ but } 0_b, 10_0b \notin \mathcal{L}_2$

$$\mathcal{L}_2 = \{swvb \mid s, v \in \{0, 1\} \wedge w \in \{0, 1, _ \}^* \wedge w \text{ cannot contain two adjacent } _ \}$$

Test with the ANTLR plugin for recognizable and not recognizable strings

MORE GRAMMARS (CONT'D)

✂ Implement a CF grammar for $\text{// } abcc, bc \in \mathcal{L}_3 \text{ but } abbcc, c \notin \mathcal{L}_3$

$$\mathcal{L}_3 = \{a^i b^l c^k \in \{a, b, c\}^* \mid i, l, k \in \mathbb{N} \wedge i + l = k\}$$

✂ Implement a CF grammar for $\text{// } 01, 011 \in \mathcal{L}_4 \text{ but } 10, 0111 \notin \mathcal{L}_4$

$$\mathcal{L}_4 = \{0^n 1^n \in \{0, 1\}^* \mid n \in \mathbb{N}\} \cup \{0^n 1^{2n} \in \{0, 1\}^* \mid n \in \mathbb{N}\}$$

Test with the ANTLR plugin for recognizable and not recognizable strings

Q + A



© Michele Pasqua, 2026

This work is licensed under the  **Creative Commons** Attribution-NonCommercial-NoDerivatives 4.0 International license. To view a copy of the license, visit:

<http://creativecommons.org/licenses/by-nc-nd/4.0>

You are free to:

 **Share** – Copy, distribute, and display the work

Under the following terms:

 **Attribution** – You must attribute the work in the manner specified by the author or licensor

 **NonCommercial** – You may not use this work for commercial purposes

 **NoDerivatives** – If you alter, transform, or build upon this work, you may not distribute the modified material

For any reuse or distribution, you must make clear to others the license terms of this work. Any of these conditions can be waived if you get permission from the copyright holder. Your fair use and other rights are in no way affected by the above.